

# Do Script Python ao Kubernetes na AWS

## A Anatomia das Conexões: Como Cada Peça se Comunica

### Estudo de Caso Prático: API de Analytics do Coworking Space

**Autor:** Eugênio | **Público-Alvo:** Analistas de Dados, Desenvolvedores e Iniciantes em Nuvem | October 2, 2026

#### Prefácio: O Enigma da Conectividade em Nuvem

Quando vemos um diagrama de arquitetura de nuvem pela primeira vez, ele pode parecer uma coleção de caixas isoladas com nomes misteriosos: Docker, CodeBuild, ECR, EKS, Pods, Services. O segredo para dominar DevOps e entender a infraestrutura moderna é responder a uma pergunta simples: **“Como o componente A conversa com o componente B?”**. Quem autentica quem? Qual porta é utilizada? Como um contêiner encontra um banco de dados sem saber o IP fixo dele?

Este documento expande nossa jornada técnica anterior, detalhando a **flação invisível** que conecta cada ferramenta da AWS e cada recurso do Kubernetes, acompanhado de fluxogramas com legendas detalhadas de funções e protocolos.

## 1 O Mapa Mental dos Conceitos (Analogias do Mundo Real)

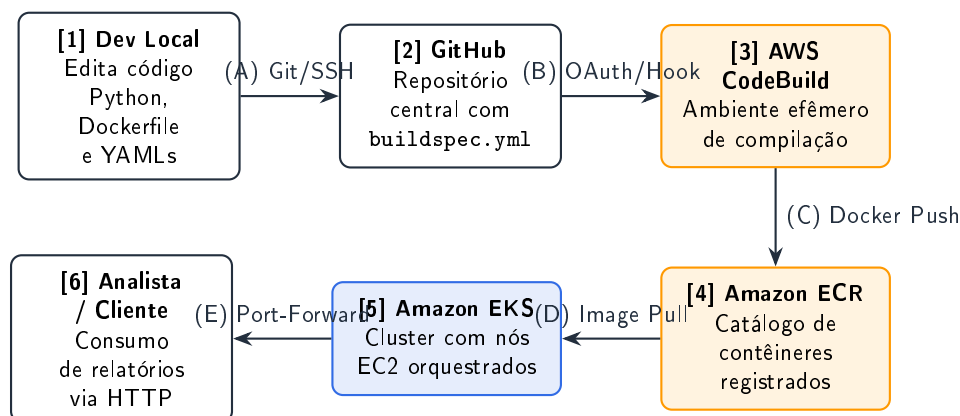
Para quem vem do ecossistema de dados (Python, SQL, Pandas), veja a correspondência direta de cada tecnologia:

- **Docker (A Marmita Lacrada):** É um ambiente virtual auto-suficiente. Ele carrega seu código Python, suas bibliotecas e o sistema operacional mínimo para garantir que o script rode idêntico em qualquer servidor do planeta, eliminando o clássico “na minha máquina funciona”.
- **AWS ECR (O Almojarifado de Imagens):** Repositório privado da AWS onde as imagens Docker empacotadas são guardadas com tags de versão (1.0.0, latest).
- **AWS CodeBuild (A Linha de Montagem / CI):** Servidor descartável na nuvem que, a cada novo código no GitHub, compila a imagem Docker e a envia ao ECR sem que você precise gastar a CPU do seu computador pessoal.
- **Kubernetes / EKS (O Maestro da Orquestra):** Gerencia os servidores e contêineres. Se o pod da aplicação cair por consumo excessivo de memória, o Kubernetes o recria imediatamente sem indisponibilidade para os usuários.
- **Helm (O Pip da Infraestrutura):** Em vez de escrever centenas de linhas de configuração manual para instalar um banco PostgreSQL, o Helm instala tudo com uma única instrução pronta da comunidade.

## 2 Fluxogramas com Legendas de Funções e Conectividade

### 2.1 Fluxograma 1: A Linha de Montagem e Desdobramento (Pipeline CI/CD)

O diagrama a seguir rastreia o ciclo de vida do código, identificando o canal e o protocolo de cada conexão:

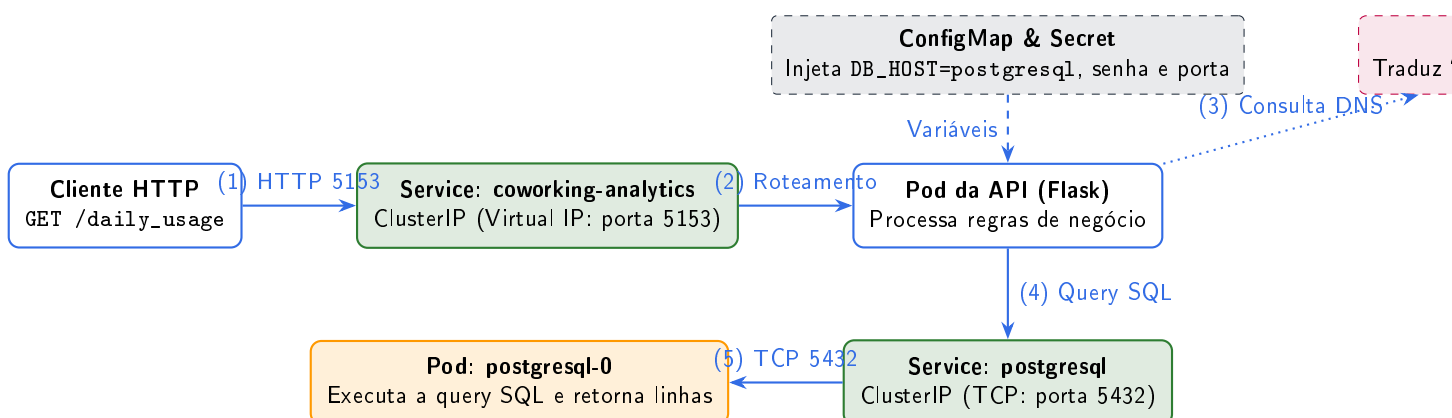


### Legenda da Linha de Montagem (Como Cada Peça se Conecta)

| Ref. | De → Para          | Protocolo / Mecanismo         | O que acontece nesta conexão?                                                                   |
|------|--------------------|-------------------------------|-------------------------------------------------------------------------------------------------|
| (A)  | Dev → GitHub       | Git over HTTPS (porta 443)    | O desenvolvedor envia os códigos do app e o arquivo <code>buildspec.yml</code> para a nuvem.    |
| (B)  | GitHub → CodeBuild | Webhook / GitHub App          | O CodeBuild detecta a nova versão e baixa o repositório para sua máquina de compilação.         |
| (C)  | CodeBuild → ECR    | Docker Login + HTTPS (443)    | O CodeBuild obtém um token com <code>aws ecr get-login-password</code> e envia a imagem gerada. |
| (D)  | EKS → ECR          | IAM Role do Nó (HTTPS 443)    | O nó do EKS autentica na AWS e faz o download seguro da imagem para iniciar os contêineres.     |
| (E)  | EKS → Analista     | Túnel TCP (Port-Forward 5153) | O analista acessa a API interna através de um túnel seguro aberto pelo <code>kubectl</code> .   |

## 2.2 Fluxograma 2: A Rede Interna do Kubernetes e o Caminho dos Dados

O diagrama abaixo detalha como a requisição de analytics viaja dentro do cluster até o banco de dados:



## Legenda da Rede Interna (Como o App Encontra o Banco)

| Passo | Componentes             | Papel na Arquitetura | Detalhes Técnicos da Conexão                                                                                      |
|-------|-------------------------|----------------------|-------------------------------------------------------------------------------------------------------------------|
| (1)   | Cliente → Service App   | Porta de Entrada     | A requisição bate no <b>Service</b> , que tem um IP virtual fixo e nunca muda.                                    |
| (2)   | Service App → Pod Flask | Balanceador de Carga | O <b>Service</b> usa o seletor ( <code>app=coworking-analytics</code> ) para repassar a chamada ao Pod saudável.  |
| (3)   | Pod Flask → CoreDNS     | Resolução de Nome    | O Flask não sabe o IP do banco; ele pergunta ao DNS interno do k8s onde fica o servidor <code>postgresql</code> . |
| (4)   | Pod Flask → Service DB  | Conexão SQLAlchemy   | O app abre uma sessão PostgreSQL autenticando com usuário e senha recebidos pelo <b>Secret</b> .                  |
| (5)   | Service DB → Pod DB     | Execução do Banco    | O PostgreSQL executa o <code>SELECT Date(created_at), Count(*) FROM tokens GROUP BY ...</code> e devolve o JSON.  |

## 3 Como os Componentes se Conectam: O Fio Condutor da Arquitetura

Para entender profundamente como a infraestrutura opera, vamos analisar a conexão entre cada par de peças:

### 3.1 1. Como o Desenvolvedor se conecta ao Cluster EKS

- **O Elo de Ligação:** O arquivo local de configuração `~/.kube/config`.
- **Como funciona:** Quando você digita `kubectl get pods`, o binário do `kubectl` não conversa magicamente com o servidor. Ele lê o arquivo de configuração, descobre a URL do *API Server* na AWS e executa em segundo plano o comando `aws eks get-token`. Esse token é assinado pelo serviço de identidade da AWS (STS) com base no seu usuário IAM. O Kubernetes valida a assinatura do token e permite a visualização dos recursos.

### 3.2 2. Como o CodeBuild se conecta ao Registro de Imagens (ECR)

- **O Elo de Ligação:** Chaves do IAM e o comando `docker login`.
- **Como funciona:** Dentro do arquivo `buildspec.yml`, executamos:

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 467369438218.dkr.ecr.us-east-1.amazonaws.com
```

A máquina virtual do CodeBuild utiliza sua *IAM Service Role* para solicitar à AWS uma senha temporária criptografada de acesso ao ECR. Com essa credencial, o comando `docker push` ganha autorização para gravar a imagem `coworking-analytics:1.0.0`.

### 3.3 3. Como a Aplicação Python encontra o Banco sem saber o IP (Service Discovery)

- **O Elo de Ligação:** O sistema de DNS embutido do Kubernetes (**CoreDNS**).
- **Como funciona:** No mundo dos contêineres, os pods morrem e nascem o tempo todo, ganhando IPs dinâmicos diferentes a cada reinício. Como o script Python se conecta a um banco cujo IP muda?

A resposta é o objeto **Service**. Criamos um serviço com o nome exato `postgresql`. O Kubernetes atribui a esse serviço um nome de domínio interno: `postgresql.default.svc.cluster.local`. No nosso **ConfigMap**, declaramos a variável de ambiente:

```
DB_HOST: "postgresql"
```

Quando o driver `psycopg2` do Python tenta conectar em `postgresql:5432`, o Kubernetes intercepta a requisição e a encaminha automaticamente para o pod do banco de dados correto.

### 3.4 4. Como os Segredos (Senhas) chegam de forma segura ao Python

- **O Elo de Ligação:** Os objetos `ConfigMap` e `Secret`.
- **Como funciona:** Dados não-sensíveis (como nome do banco e porta) foram configurados no `ConfigMap`. Já a senha do PostgreSQL foi armazenada em um `Secret` protegido. No arquivo `deployment.yaml`, usamos a instrução `secretKeyRef` para ligar a chave do secret diretamente a uma variável de ambiente chamada `DB_PASSWORD`. O Python simplesmente lê `os.environ["DB_PASSWORD"]`, sem nunca salvar a senha em arquivos de texto ou código-fonte.

—

## 4 A Crônica Detalhada das Etapas (O Que Fizemos e Por Quê)

A seguir, a narrativa completa de cada etapa executada no projeto:

### 4.1 Etapa 1: O Ponto de Partida e o Mistério da AWS sem Chaves

**O desafio:** Ao rodar `aws sts get-caller-identity` no terminal local, recebemos o erro informando que o comando não existia. Após instalá-lo, surgiu a mensagem *“Unable to locate credentials”*.

**A explicação:** O comando **AWS CLI** é apenas o canal de comunicação com os datacenters da AWS. Mas antes de qualquer ação, a AWS exige comprovação de identidade. As variáveis exportadas no terminal (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` e `AWS_SESSION_TOKEN`) agem como um crachá de segurança com foto e chave magnética. Com elas ativas, a AWS respondeu imediatamente reconhecendo a nossa conta.

### 4.2 Etapa 2: Erguendo a Cidade do Kubernetes com eksctl

**O desafio:** Executamos o comando de criação do cluster:

```
eksctl create cluster --name my-cluster --region us-east-1 --nodegroup-name my-nodes  
--node-type t3.small --nodes 1
```

O comando executou por 18 minutos ininterruptos.

**A explicação:** Ele não estava travado; estava provisionando uma infraestrutura completa do zero por meio de automação do CloudFormation:

1. Criou a **VPC**: uma rede privada e segura no datacenter da AWS em Virgínia.
2. Criou sub-redes e tabelas de roteamento com firewalls gerenciados.
3. Inicializou os servidores mestres de alta disponibilidade (**Control Plane**).
4. Subiu uma instância virtual EC2 (`t3.small`) como nó operário (*worker node*).

Ao término, o `eksctl` gerou a chave `~/.kube/config`, habilitando o comando `kubectl` a gerenciar o cluster a partir do terminal.

### 4.3 Etapa 3: Criando e Populando o Banco de Dados com Helm

**O desafio:** Instalamos o PostgreSQL via `helm install postgresql bitnami/postgresql` e populamos o banco com os scripts SQL de tabelas, usuários e tokens.

**A explicação:** O Helm aplicou uma receita pronta de banco de dados, configurando o armazenamento e criando uma senha aleatória no `Secret`. Quando tentamos conectar via porta local 5432, ocorreu falha de autenticação porque o computador já possuía outro PostgreSQL rodando localmente na mesma porta. Resolvemos isso injetando os scripts SQL diretamente para dentro do pod do banco na nuvem com o comando `kubectl exec`, eliminando qualquer interferência da máquina local.

### 4.4 Etapa 4: Empacotando o Python no Docker e o Bug Oculto do Flask

**O desafio:** Criamos o `Dockerfile` com `python:3.10-slim`. Nos testes dentro do pod, a aplicação quebrou com o erro:

```
ImportError: cannot import name 'url_quote' from 'werkzeug.urls'
```

**A explicação:** O arquivo `requirements.txt` pedia apenas `flask==2.2.2`. Sem versão fixada para a biblioteca dependente `Werkzeug`, o pip instalou a versão mais recente (`Werkzeug 3.0`), que removeu a função `url_quote` que o Flask esperava. Fixamos a dependência em `Werkzeug==2.2.2`, garantindo estabilidade e reprodutibilidade da imagem Docker.

### 4.5 Etapa 5: Automatizando a Nuvem com GitHub e AWS CodeBuild

**O desafio:** Criamos o arquivo de pipeline `buildspec.yml` e o vinculamos ao **AWS CodeBuild**.

**A explicação:** O CodeBuild é o robô de fábrica da AWS. A cada execução, ele sobe uma máquina efêmera, autentica no ECR, compila o `Dockerfile`, tagueia a imagem com versionamento semântico (1.0.0 e latest) e a envia ao armazém **AWS ECR**, deixando-a pronta para ser consumida pelo Kubernetes.

### 4.6 Etapa 6: O Mistério do Pod que Não Ficava Pronto (Readiness Probe)

**O desafio:** O comando `kubectl rollout status` permaneceu bloqueado com a mensagem *“1 old replicas are pending termination...”*, enquanto o novo pod ficava em `0/1 Running`.

**A explicação:** O Kubernetes possui sensores médicos chamados **Probes**. O **Readiness Probe** pergunta continuamente: *“Você já está pronto para receber requisições de clientes?”*. Nosso manifesto original testava a rota `/readiness_check`, que continha um erro de código Python ao tentar buscar um modelo `Token` inexistente, devolvendo status HTTP 500. Como o novo pod nunca dizia estar pronto, o Kubernetes recusava-se a desligar a réplica antiga para proteger os usuários. Ajustamos o probe para a rota saudável `/health_check`, e o pod alcançou o status `1/1 Ready` instantaneamente.

### 4.7 Etapa 7: O Gran Finale – Relatórios de Negócio Respondendo 100%

**O desafio:** Abrimos a porta 5153 e disparamos requisições de teste. Após adicionar parênteses em `get_daily_visits()` no `app.py`, ambas as rotas retornaram dados completos:

```
{"2023-02-07":40,"2023-02-08":202,"2023-02-09":179,"2023-02-10":158,...}
```

O microserviço consultou com sucesso os 500 usuários e tokens no PostgreSQL e entregou os relatórios analíticos em JSON formatado!

## 5 Decisões de Engenharia e Boas Práticas (Standout Topics)

### 1. Por que definir Requests e Limits de CPU e Memória?

Em análise de dados, uma consulta pesada sem filtro pode consumir toda a memória RAM e travar o servidor inteiro. No Kubernetes prevenimos isso com:

- **Requests** (cpu: 100m, memory: 64Mi): É a reserva garantida mínima para o pod funcionar.
- **Limits** (cpu: 250m, memory: 256Mi): É o teto máximo. Se o pod tentar usar mais de 256 MB de RAM, o Kubernetes impede o vazamento de memória antes que ele afete os outros contêineres do nó.

### 2. Escolha da Instância EC2: Por que a família Graviton (t4g)?

Recomendamos a instância **t4g.small**. Equipada com os processadores proprietários **AWS Graviton2** (arquitetura ARM64), ela entrega até **40% mais performance** computacional com um **custo 20% menor** em relação às instâncias x86 tradicionais, sendo perfeita para APIs web leves e relatórios analíticos.

### 3. Como Economizar Dinheiro Real na AWS?

1. **Instâncias Spot com Karpenter:** A AWS oferece servidores ociosos com até 90% de desconto. Para tarefas analíticas e ambientes de teste, usar instâncias Spot reduz dramaticamente os custos.
2. **Autoescalabilidade (HPA):** Permite que o cluster aumente o número de réplicas durante o pico de uso comercial e encolha para apenas 1 pod nos finais de semana e noites.

—

## 6 Conclusão

Ao entender como cada componente se conecta — através de redes internas, resolução DNS, autenticações IAM e orquestração Kubernetes —, a distância entre o analista que escreve a query SQL e a infraestrutura em nuvem desaparece, viabilizando soluções seguras, eficientes e escaláveis.